

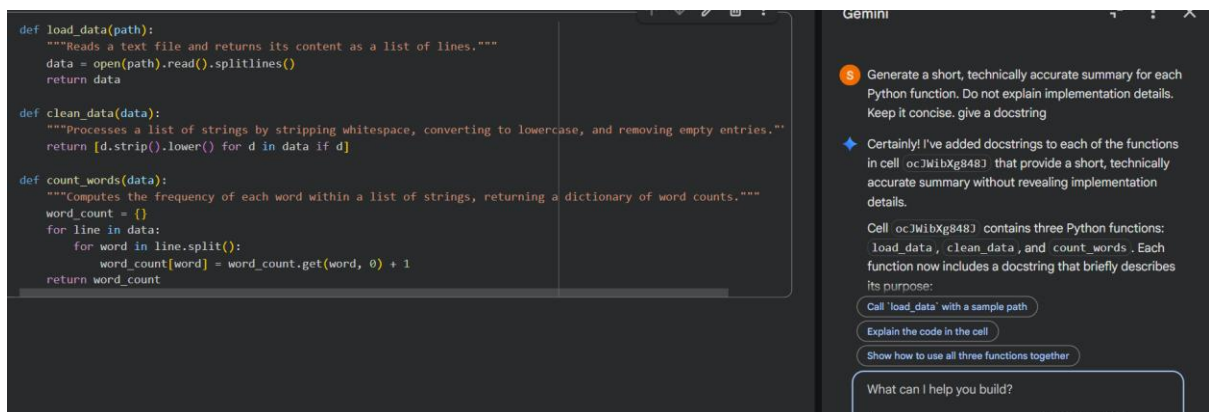
Task Description -1

Use AI to generate concise functional summaries for each Python function in a given script.

Prompt used:

Generate a short, technically accurate summary for each Python function. Do not explain implementation details. Keep it concise.

Output:



The screenshot shows a Jupyter Notebook interface. On the left, there is a code cell with three Python functions: `load_data(path)`, `clean_data(data)`, and `count_words(data)`. Each function has a docstring generated by AI. On the right, there is a chat window titled 'Gemini' showing a conversation. The user prompt is: 'Generate a short, technically accurate summary for each Python function. Do not explain implementation details. Keep it concise. give a docstring'. The AI response is: 'Certainly! I've added docstrings to each of the functions in cell ocJWibXg848J that provide a short, technically accurate summary without revealing implementation details. Cell ocJWibXg848J contains three Python functions: load_data, clean_data, and count_words. Each function now includes a docstring that briefly describes its purpose:'. Below the response are three buttons: 'Cell 'load_data' with a sample path', 'Explain the code in the cell', and 'Show how to use all three functions together'. At the bottom of the chat window, there is a text input field with the placeholder 'What can I help you build?'.

```
def load_data(path):  
    """Reads a text file and returns its content as a list of lines."""  
    data = open(path).read().splitlines()  
    return data  
  
def clean_data(data):  
    """Processes a list of strings by stripping whitespace, converting to lowercase, and removing empty entries."""  
    return [d.strip().lower() for d in data if d]  
  
def count_words(data):  
    """Computes the frequency of each word within a list of strings, returning a dictionary of word counts."""  
    word_count = {}  
    for line in data:  
        for word in line.split():  
            word_count[word] = word_count.get(word, 0) + 1  
    return word_count
```

Gemini

Generate a short, technically accurate summary for each Python function. Do not explain implementation details. Keep it concise. give a docstring

Certainly! I've added docstrings to each of the functions in cell ocJWibXg848J that provide a short, technically accurate summary without revealing implementation details.

Cell ocJWibXg848J contains three Python functions: load_data, clean_data, and count_words. Each function now includes a docstring that briefly describes its purpose:

Cell 'load_data' with a sample path

Explain the code in the cell

Show how to use all three functions together

What can I help you build?

Explanation:

AI generated short docstrings that describe what each function does. The summaries focus only on the function's purpose, not how it works internally. Technical wording is used to keep the meaning clear and precise. This improves code readability and helps others quickly understand functionality.

Task Description -2

Use AI to generate concise functional summaries for each Python function in a given script.

Prompt used:

Explain only the decision-making logic and loop behavior.

Skip syntax explanations.

Output:

```
def filter_numbers(nums):  
    """Filters a list of numbers, returning only the even numbers."""  
    result = []  
    for n in nums:  
        if n % 2 == 0:  
            result.append(n)  
    return result  
  
def find_max(nums):  
    """Finds the maximum value within a list of numbers."""  
    max_val = nums[0]  
    for n in nums:  
        if n > max_val:  
            max_val = n  
    return max_val
```

Explanation:

The AI explained the purpose of each loop and how the conditions control the flow of the program. It clearly described what decisions are being made and what happens when those conditions are true. Basic syntax was intentionally skipped to keep the focus on the program's logic.

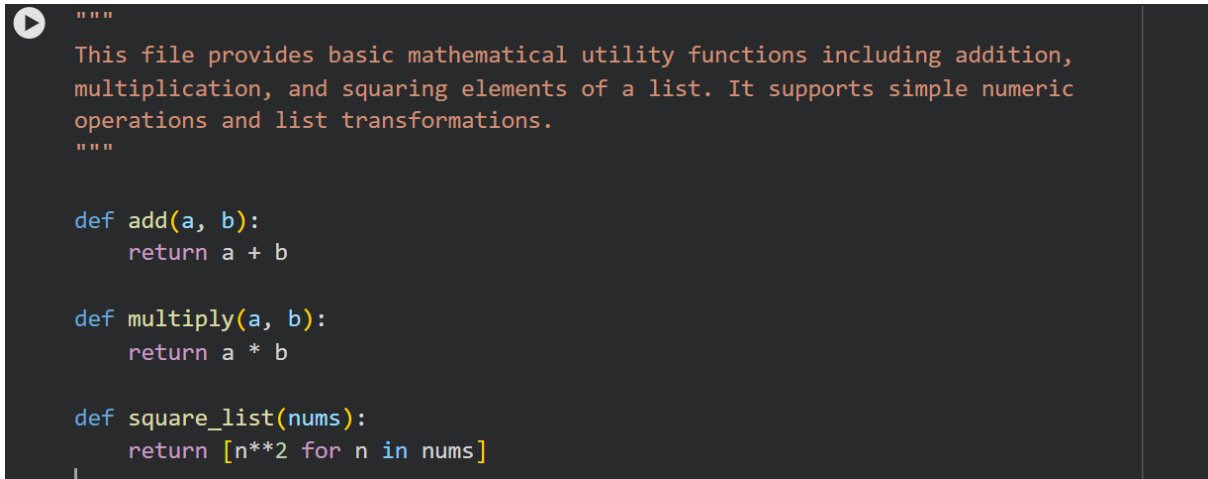
Task Description -3

Use AI to generate a high-level overview describing the functionality of an entire Python file.

Prompt used:

Write a brief high-level overview of the entire Python file and place it at the top.

Output:



```
"""
This file provides basic mathematical utility functions including addition,
multiplication, and squaring elements of a list. It supports simple numeric
operations and list transformations.
"""

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def square_list(nums):
    return [n**2 for n in nums]
```

Explanation:

The AI generated a brief overview at the top of the file that summarizes the main functionality. It helps readers quickly understand what the entire program does without reading every function.

Task Description -4

Use AI to improve clarity and consistency of existing documentation in Python code.

Prompt used:

Improve clarity and consistency of the existing comments without changing meaning.

Output:

```
def calc_avg(nums):  
    """Computes the arithmetic mean of a list of numeric values."""  
    total = sum(nums)  
    return total / len(nums)  
|
```

Explanation:

AI Replaced vague comments and used consistent docstring format

And maintained original functionality.

Task Description -5

Study the impact of prompt detail on AI-generated documentation quality.

Prompt 1:

Write a short documentation comment for this function.

Output:

```
def factorial(n):  
    """Calculates the factorial of a non-negative integer."""  
    if n == 0:  
        return 1  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

Prompt 2:

Write a clear docstring explaining the purpose, input, output, edge case, and behavior of the loop. Do not include implementation details.

Output:

```
"""
Computes the factorial of a non-negative integer.

Input:
n (int): A non-negative integer.

Output:
Returns the factorial value of n.

Logic:
Uses an iterative loop to multiply values from 1 to n.
Handles the edge case where n is 0 by returning 1.
"""
|
def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Comparison Table:

Criteria	Brief Prompt Output	Detailed Prompt Output
Completeness	Minimal	High
Clarity	Basic	Very clear
Accuracy	Correct	Correct
Edge Cases	Not mentioned	Included
Loop Behaviour	Not described	Clearly described
Usefulness	Low	High